2026-8-10 Linux信号--中断

- 1. 中断的基本模型
  - 1.1 什么是中断
    - 1.1.1 中断是异步的硬件事件通知
    - 1.1.2 中断的核心动作
  - 1.2 硬件中断的组成
    - 1.2.1 设备、中断控制器与 CPU
    - 1.2.2 轮询与中断驱动
    - 1.2.3 为什么中断必须进入内核
- 2. 一次硬件中断的处理流程
  - 2.1 从设备就绪到中断返回
  - 2.2 CPU 保存什么现场
  - 2.3 中断描述符表与处理入口
  - 2.4 中断服务程序做什么
- 3. 时钟中断与调度
  - 3.1 时钟源怎样产生周期中断
  - 3.2 时钟中断为什么重要
  - 3.3 经典 Linux 0.11 的时间片模型
  - 3.4 系统内核为什么能持续存在
- 4. 中断、异常、软中断与信号
  - 4.1 三类内核入口
  - 4.2 硬件中断是外部异步事件
  - 4.3 异常是当前指令导致的同步事件
  - 4.4 软中断与系统调用入口
  - 4.5 信号不是硬件中断
- 5. 系统调用与特权级切换
  - 5.1 从库函数到系统调用入口
  - 5.2 系统调用表与分派
  - 5.3 CPU 特权级
  - 5.4 栈切换与地址空间
  - 5.5 内核怎样返回用户态
- 6. 中断机制的逻辑闭环
  - 6.1 从设备事件到原任务恢复
  - 6.2 中断与系统调用、异常的区别
  - 6.3 关键结论

1. 中断的基本模型

1.1 什么是中断

1.1.1 中断是异步的硬件事件通知

中断是外部设备或 CPU 内部事件打断当前指令流、使 CPU 转去执行特定处理程序的一种机制。处理完成后，CPU 再恢复被打断的现场，继续原来的工作。

“异步”表示中断可能出现在当前程序执行的任意位置，程序不能提前规定它只会发生在某一条语句之后。这一点与普通函数调用不同：

```
普通函数调用：当前代码主动调用目标函数
中断：外部事件在不可预知的位置打断当前代码
```

中断机制使 CPU 不必持续查询每个设备是否就绪，从而把“等待事件”转化为“事件主动通知”。

1.1.2 中断的核心动作

一次中断至少包含四件事：

1. 硬件事件发生。
2. CPU 暂停当前任务，保存后续恢复所需的现场。
3. CPU 根据中断号找到处理入口并执行中断服务程序。
4. 处理完成后恢复现场，继续原任务。

因此，中断不是“设备直接执行操作系统代码”，而是“设备通知 CPU，CPU 再按预先配置的路径进入内核处理”。

1.2 硬件中断的组成

1.2.1 设备、中断控制器与 CPU

长图把硬件中断路径拆为三个层次：

```
键盘 / 网卡 / 磁盘 / 时钟源
                |
```

```
        v
中断控制器
        |
        v
       CPU
```

各角色的作用如下：

- **外部设备**：事件就绪时发出中断请求，例如键盘有输入、网卡收到数据、磁盘 I/O 完成。
- **中断控制器**：汇聚多个设备的请求，确定优先级和中断号，再通知 CPU。现代系统通常使用 APIC 等控制器。
- **CPU**：识别中断请求，保存必要现场，根据中断号进入对应处理入口。

设备请求编号与 CPU 使用的中断向量在具体硬件上可能经过映射，不应当把设备线号、IRQ 号和中断向量号视为永远相同。

1.2.2 轮询与中断驱动

CPU 发现设备事件有两种基本模型：

模型	工作方式	特点
轮询	CPU 周期性读取设备状态	实现简单，但大量查询可能是无效工作
中断	设备就绪时主动通知 CPU	平时不必反复检查，响应及时

如果操作系统完全靠轮询键盘、网卡和磁盘，会浪费大量 CPU 时间。中断使设备事件到来时内核才介入，因此是现代操作系统管理 I/O 的基础机制。

1.2.3 为什么中断必须进入内核

设备寄存器、页表、中断控制器和中断描述符表都属于受保护资源。用户态程序通常只能通过系统调用请求内核操作这些资源，不能任意修改中断入口，否则任何进程都可能劫持系统控制流或获得内核权限。

因此，中断发生的瞬间必须切换到高权限执行环境，由内核完成设备处理和必要的安全控制。

2. 一次硬件中断的处理流程

2.1 从设备就绪到中断返回

可以把长图的七步流程概括如下：

```
1. 外设事件就绪
2. 外设向中断控制器发起中断请求
3. 中断控制器通知 CPU
4. CPU 获知中断并取得中断号或向量号
5. CPU 与内核保存被打断任务的硬件现场
6. CPU 根据向量进入对应中断处理程序
7. 处理完毕，恢复现场并继续被打断的任务
```

这里的“继续原任务”是通常情况。中断处理也可能唤醒等待该设备的进程、记录时间、触发调度，或者在返回用户态前向当前进程递送信号。

2.2 CPU 保存什么现场

中断打断的是 CPU 当前正在执行的指令流，因此必须先保存能够恢复该指令流的信息：

- 下一条要执行的指令位置。
- 当前栈指针。
- 相关通用寄存器和状态寄存器。
- 必要时保存中断前的特权级和栈信息。

CPU 硬件通常会自动保存一部分最小现场；内核的中断入口随后再保存其余寄存器，构造完整的内核执行上下文。

如果不保存现场，中断处理程序覆盖寄存器以后，就无法回到原程序被中断的位置继续执行。

2.3 中断描述符表与处理入口

CPU 必须知道某个中断号对应哪段处理代码。x86 传统模型使用中断描述符表（Interrupt Descriptor Table，IDT）：

```
中断向量 n  ->  IDT[n]  ->  中断处理入口
```

可以先把 IDT 理解为一个“按中断号索引的描述表”，但不能简单等同于普通 C 数组：

- 表项中保存处理入口地址。
- 表项还包含权限、门类型以及与特权级切换有关的属性。
- IDT 由操作系统在启动阶段建立和加载。
- 用户程序不能直接修改 IDT。

长图用类似下面的形式帮助理解：

```
typedef void (*handler_t)(void);
handler_t IDT[NUM];
```

这是概念模型。真实 x86 IDT 表项是指向代码段偏移、选择子和属性共同组成的描述符，并非只保存一个裸函数指针。

2.4 中断服务程序做什么

中断服务程序（Interrupt Service Routine，ISR）根据中断来源完成必要工作：

- 读取设备状态并清除中断原因。
- 处理键盘、网络、磁盘等设备事件。
- 更新内核计数器和时间信息。
- 唤醒等待该设备的任务。
- 通知中断控制器本次中断处理完成。

中断上下文通常要求短小，因为 CPU 正在承担最高优先级的服务路径。耗时工作应在线程上下文或延迟处理机制中完成，不能把中断上下文当作可以随意睡眠的普通线程。

3. 时钟中断与调度

3.1 时钟源怎样产生周期中断

下图模型说明时钟中断来自硬件时钟源：

```
晶振产生高频脉冲
        |
        v
时钟计数器 / 可编程分频器
        |
```

```
      v
周期性时钟中断
```

晶振频率远高于操作系统需要处理时钟中断的频率，因此硬件通过计数器或分频器得到周期性中断，例如每隔 1 ms 或 10 ms 触发一次。

图中的应用处理器标称 3.20 GHz，表示晶振和倍频后得到的基本时钟速度；内核不会让 CPU 每个时钟周期都执行一次操作系统调度。

3.2 时钟中断为什么重要

时钟中断为操作系统提供了“从运行中的用户任务收回控制权”的稳定机会。即使某个用户程序当前正在执行长循环，只要时钟中断到来，CPU 仍会进入内核。

内核可以借此：

- 累计经过的时间。
- 更新当前任务已使用的时间片。
- 检查是否应该切换任务。
- 处理超时、定时器等事件。

因此，操作系统的调度并不依赖用户进程“主动让出 CPU”。抢占式调度必须依赖时钟中断或其他外部中断提供执行机会。

3.3 经典 Linux 0.11 的时间片模型

长图使用 Linux 0.11 帮助理解调度过程。每个任务的 `task_struct` 中有：

```
long counter;
```

`counter` 表示当前任务还可使用的时间片。时钟中断到来时会调用类似下面的处理逻辑：

```
void do_timer()
{
    // 为当前任务递减时间片
    // 若时间片耗尽，则触发 schedule()
}
```

调度时可能重新计算时间片并切换任务：

```
(*p)->counter = ((*p)->counter >> 1) + (*p)->priority;
switch_to(next);
```

这是早期 Linux 的示例代码，不是现代 Linux 调度器的实现。现代内核使用 CFS 等调度机制，并支持动态时钟与高精度定时器等方案；但其共同前提仍然是：内核需要周期性或可编程的硬件时间事件来重新获得控制权。

3.4 系统内核为什么能持续存在

操作系统本身不是像普通用户进程那样始终占用一个 CPU 执行用户级死循环。内核代码平时不运行，只有在系统调用、异常或中断发生时，CPU 才从用户代码切入内核。

可以这样理解：

- 用户任务运行时，内核处于等待事件的空闲状态。
- 中断到达时，CPU 跳入内核中断入口。
- 内核执行 ISR、调度器或系统调用实现。
- 处理完成后返回用户任务，或者切换到另一个任务。

“操作系统是建立在中断处理机制上的内核代码”这一说法强调：内核依靠可控的入口在用户程序和设备事件之间取得执行权。

4. 中断、异常、软中断与信号

4.1 三类内核入口

长图把进入内核的事件分为若干类别：

类别	触发来源	与当前指令的关系	典型例子
硬件中断	外设或时钟	异步	键盘、网卡、定时器
软件触发内核入口	当前指令主动触发	同步	`int 0x80`、`syscall`
异常	CPU 执行指令时检测到故障	同步	除零、缺页、非法指令

它们的共同点是都需要从用户执行流转入内核；不同点是触发来源、发生时机和返回语义不同。

4.2 硬件中断是外部异步事件

硬件中断由 CPU 外部设备或可编程时钟触发，并不由当前 C 语言语句直接决定发生时刻。

典型链路是：

```
外设就绪 -> 中断控制器 -> CPU -> IDT -> 内核 ISR
```

即使进程什么也不做，只要硬件事件到来，CPU 也可能被中断。

4.3 异常是当前指令导致的同步事件

异常由 CPU 在执行当前指令时检测到异常条件而产生，例如：

- 除零：算术异常。
- 解引用非法地址：MMU/页表检查失败。
- 缺页：目标页面暂时不在物理内存中。
- 非法指令：CPU 无法执行当前编码。

异常与触发指令直接相关。内核处理以后，可能修复现场并重新执行故障指令，也可能终止进程。

4.4 软中断与系统调用入口

`int 0x80` 是 x86 32 位系统经典的系统调用入口，它通过 IDT 中的门描述符进入内核。现代 x86-64 Linux 通常使用 `syscall` 指令；`syscall` 使用 CPU 中预先配置的系统调用入口，而不是从 IDT 中按向量取表项。

两者都能让 CPU 从用户态切换到内核态，但底层入口机制不能混为一谈。

“软中断”这一术语在课程中可以表示由软件主动触发的中断路径，但 Linux 内核里还有专门的 deferred interrupt 机制也叫 softirq。二者不是完全相同的概念：

- 本节的“软中断”主要指 `int 0x80` 一类的软件触发内核入口。

- Linux `softirq` 指中断处理程序中延后执行的软件中断机制。

学习时要结合上下文区分。

4.5 信号不是硬件中断

信号与中断有相似的“打断当前执行流并处理事件”的外观，但作用层级不同：

- 硬件中断通知 CPU，由内核进入 ISR。
- 信号通知进程，由操作系统保存并在后续递送给目标进程。
- 硬件中断处理发生在内核态；自定义信号 handler 通常回到用户态执行。

所以可以把信号理解为“面向进程的软件事件通知”，但不能把产生信号等同于产生一次 CPU 硬件中断。

5. 系统调用与特权级切换

5.1 从库函数到系统调用入口

以 `read()` 为例，调用链大致为：

```
用户调用 read()
    |
    v
C 库包装函数准备参数和系统调用号
    |
    v
执行 syscall / int 0x80
    |
    v
CPU 切换到内核态
    |
    v
内核按系统调用号分派
    |
    v
执行 sys_read() 等内核函数
    |
    v
返回用户态
```

在图示的 x86 32 位模型中，常见做法是把系统调用号放入 `eax`，把参数放入约定的寄存器，再执行 `int 0x80`。具体寄存器约定随体系结构和调用 ABI 变化。

5.2 系统调用表与分派

内核维护系统调用函数表，概念上可以写成：

```
fn_ptr sys_call_table[] = {
    sys_setup,
    sys_exit,
    sys_fork,
    sys_read,
    sys_write,
    // ...
};
```

系统调用入口取得系统调用号后，按编号查找对应函数：

```
int sysnumber = eax;
sys_call_table[sysnumber](/* 参数 */);
```

因此，系统调用机制本质上是“用户触发受控入口 -> 内核保存现场 -> 按编号分派 -> 执行内核函数 -> 返回用户态”。系统调用号选择具体功能，参数由调用 ABI 规定。

5.3 CPU 特权级

x86 特权级使用环编号：

- Ring 0：内核态，权限最高。
- Ring 3：用户态，权限较低。
- Ring 1、2 通常不被常规 Linux 用户程序使用。

相关概念：

- **CPL**（Current Privilege Level）：当前代码正在运行的特权级，通常由代码段寄存器低两位表示。
- **DPL**（Descriptor Privilege Level）：描述符允许访问的特权级。
- **RPL**（Requested Privilege Level）：请求者通过选择子表达的特权级。

在经典 `int 0x80` 路径中，相应门描述符的 DPL 允许用户程序发起这次受控转换；x86-64 的 `syscall` 路径则使用单独配置的系统调用入口。两条路径最终都会把 CPL 从 3 切换到 0。权限检查和模式切换由硬件参与，不能仅靠用户程序修改一个普通变量完成。

5.4 栈切换与地址空间

用户态进入内核态时，不能继续无条件使用用户栈，因为内核需要独立的受保护调用栈。经典 x86 模型会在任务状态段或相关任务上下文中记录内核栈的 `SS0`、`ESP0`，CPU 在特权级切换时据此切换栈。

经典 32 位 Linux 图示采用：

- 用户空间约为 `[0, 3 GiB)`。
- 内核空间约为 `[3 GiB, 4 GiB)`。
- 每个进程通常有独立的用户页表。
- 内核空间的页表映射由所有任务共享，使 CPU 无论切换到哪个进程，都能找到内核代码和内核数据。

现代 64 位 Linux 的地址范围和页表组织不同，但“内核映射必须始终可达、用户区域按进程隔离、内核拥有更高权限”的原则仍然成立。

5.5 内核怎样返回用户态

系统调用处理结束后，内核根据保存的现场恢复用户寄存器和用户栈，并执行适合体系结构的返回指令，使 CPU 从 Ring 0 回到 Ring 3。

返回用户态之前，内核还会检查：

- 是否有待递达信号。
- 是否需要执行信号处理函数。
- 是否需要切换到其他任务。
- 是否可用用户最初期望的返回值继续执行。

这说明中断、系统调用和异常不仅是“进入内核”的入口，也是操作系统检查时间片、信号和调度需求的重要边界。

6. 中断机制的逻辑闭环

6.1 从设备事件到原任务恢复

```
设备或时钟产生事件
        |
        v
中断控制器通知 CPU
        |
        v
CPU 获取中断向量并切换特权级
        |
        v
保存现场，按 IDT 进入内核
        |
        v
中断服务程序处理事件
        |
        +-- 唤醒任务
        +-- 更新时间片并决定调度
        +-- 在返回用户态前检查信号
        |
        v
恢复现场
        |
        v
继续原任务或切换到新任务
```

6.2 中断与系统调用、异常的区别

维度	硬件中断	系统调用	异常
发起者	外设、时钟等硬件	当前软件指令	CPU 检测当前指令
时机	异步	同步	同步
主要目的	响应设备事件	请求内核服务	处理执行故障
入口	中断向量	`syscall` / `int 0x80`	异常向量
典型结果	处理 I/O、调度、计时	完成内核服务后返回	修复现场或终止进程

6.3 关键结论

- 中断是异步硬件事件，核心价值是让 CPU 在事件到来时再处理，而不必持续轮询。
- 中断控制器负责汇聚设备请求，CPU 根据中断向量进入受保护的内核入口。

- IDT 保存中断入口及其权限属性，不是普通用户可直接修改的函数指针数组。
- CPU 与内核必须保存现场，处理结束以后才能恢复被打断的程序。
- 时钟中断为操作系统提供抢占、计时和调度的执行机会。
- 经典 Linux 0.11 使用 `counter` 时间片和 `do_timer`、`schedule` 解释调度，现代 Linux 已采用不同的调度与定时实现。
- 系统调用通过软件触发、特权级切换和系统调用表进入内核，是受控的软件入口。
- 硬件中断是异步的，异常是由当前指令触发的同步事件，二者不能混为一谈。
- 信号模拟了中断式的事件通知体验，但它通知的是进程，不是直接打断 CPU 的硬件中断。